XRPL COMMONS / 36-HOUR HACKATHON / PRODUCT SPECIFICATION

Shellfish
Agentic Credit Gateway for XRP Ledger

Portable credit evidence · Lender-owned underwriting · Policy compatibility · Deterministic authorization · Native XRPL loan execution

PRODUCT DECISION

Build an interoperability and authorization layer around existing XRPL lending primitives (XLS-65/66). No secondary micro-bonds, debt trading marketplace, proprietary lending pool, or universal credit score.

FIELD	DETAIL
Document	Product Requirements Document (PRD)
Version / date	1.1 / 24 October 2026 / Two-engineer allocation update
State	Agreed concept; build-ready hackathon scope; technical assumptions labeled
Product	Shellfish — Agentic Credit Gateway
Team workstreams	4 members; only 2 implement code: You (XRPL, Shellfish authorization/signing, UI, integration) and Gita (evidence, lender policy adapters, Nick compatibility, exposure registry). Nick (policy/product and pitch) and Shell (security/QA and pitch) are non-coding contributors.
Primary track	Lending & Borrowing; agentic-finance fit
Prototype asset	RLUSD-like demo asset or supported XRPL test asset; no live institutional funds

Decision in one line

An accountable business can delegate bounded borrowing authority to an AI agent; participating lenders keep their own underwriting rules; Shellfish verifies the exact approved loan before a required signer participates in XRPL execution.

Prepared as an implementation contract and judge-facing narrative, not a production credit-policy manual or legal opinion.

1 Executive summary & product thesis

XRPL is introducing ledger-native fixed-term uncollateralized lending through pooled Single Asset Vault capital and LoanBroker/Loan objects. The core ledger primitives do not solve identity provenance, external underwriting integration, reusable evidence, agent policy compatibility or business-defined delegated borrowing limits. Shellfish fills that application-layer gap; it does not replace lending risk management or XRPL consensus. [R1–R4]

THE THESIS

AI may discover, compare and propose a loan. The lender independently decides what credit to offer. Deterministic policy checks and required signatures decide whether this exact proposed action is authorized. The validated ledger result—not an AI response—decides whether it executed.

1.1 The product in 30 seconds

A verified company authorizes an AI agent to borrow up to a defined maximum, for an approved purpose, term and cost. Gita packages relevant, attributable credit evidence. Two lender adapters independently evaluate that evidence using their own underwriting policies; one may approve, another may decline. Nick normalizes eligible offers and computes the overlap with the company mandate. Shellfish checks the exact proposed LoanSet transaction, reserves lender-specific headroom, controls one required signature and produces an auditable decision. After XRPL validation, the system reconciles state and records repayment performance back to Gita.

1.2 Product boundaries

WE ARE	WE ARE NOT
An underwriting-policy integration layer	A lender, bank or capital provider
A portable evidence and known-exposure interface	A universal FICO or mandatory scoring oracle
A lender-offer compatibility engine	An AI making final credit decisions for lenders
A deterministic authorization and controlled-signing service	A new XRPL consensus rule or unrestricted transaction signer
An adapter to native XRPL lending where enabled	A secondary debt / micro-bond marketplace

1.3 Primary user story

As a treasury or procurement team, I want an AI agent to seek a short-term RLUSD working-capital loan across participating providers while I retain legal accountability, see comparable terms, and know it cannot borrow outside the signed mandate I approved.

1.4 Non-negotiable invariants

- The agent is not itself the legal borrower; a verified principal remains accountable.
- Evidence and reputation are not a lender commitment; lender approval is issuer- and facility-specific.
- An AI output cannot change policy, limits, signatures, or authorized transaction bytes.
- Known exposure is only the exposure covered by contributing sources; it is not global debt.
- Authorization is not settlement; only validated XRPL state establishes on-chain success.

- Defaults, restructurings and charge-offs never automatically restore access to new borrowing.

2 Users, jobs-to-be-done & scope

ACTOR	JOB / INCENTIVE	MVP REPRESENTATION
Borrowing business / principal	Obtain working capital safely through delegated software	Demo company with signer and legal-principal ID
Borrowing AI agent	Discover/select a suitable approved loan without master-key access	One agent sends signed intent; can choose offered terms
Lender / Loan Broker	Apply familiar credit policy; control exposure and pricing	Two deterministic policy adapters; one configured broker
Evidence issuer	Provide attributable, versioned performance records	Simulation labeled issuer + evidence receipts
Risk / audit operator	See who authorized what, why, and what actually settled	Readable decisions, reason codes, transaction explorer link

2.1 Goals

- G1 — Reuse a consistent borrower evidence bundle across at least two independent lender policies.
- G2 — Prove one agent can obtain an eligible loan without receiving unrestricted signing authority.
- G3 — Demonstrate deterministic accept/deny on amount, APR, term, asset, counterparty, identity, nonce and freshness.
- G4 — Demonstrate one native lending transaction on a compatible development network when supported; otherwise show an honestly labeled fallback.
- G5 — Emit full authorization and settlement receipts with auditable versioned references.

2.2 Explicit non-goals

- No secondary-market claims, NFTs representing debts, bond trading, factoring, or bilateral receivable settlement.
- No original lender liquidity pool, new consensus primitive, collateral liquidation or chain-spanning settlement.
- No proprietary cross-institution universal score, automated credit approval, real-world KYC provider integration, collections, or licensed lending service.
- No production custody, regulatory certifications, ZK credit passports, x402 orchestration, or globally complete liability registry.
- No self-modifying LLM policy engine or unconstrained agent-to-agent negotiation.

2.3 MVP versus future release

AREA	P0 — HACKATHON	P1+ — AFTER HACKATHON
Credit evidence	Demo issuer, provenance, known debts, expiry	Trusted institutional issuers, selective disclosure, attestations
Lender policy	Two deterministic simulated rulesets	Integration SDK for institutional LOS/risk systems

AREA	P0 — HACKATHON	P1+ — AFTER HACKATHON
Negotiation	Comparison and selection from approved offers	Bounded counteroffers within explicit mandates
Authorization	Signed demo mandate, stateful nonce, controlled signer	HSM/MPC, revocation feed, policy version lifecycle
Loan execution	One compatible XRPL Devnet loan + repay if possible	Broader asset/network support, servicing integration
Exposure	Lender-specific atomic facility reservation	Federated multi-provider exposure reconciliation

3 End-to-end product experience

3.1 Happy path (illustrative)

STEP	SYSTEM BEHAVIOR
1 Register borrower	Map business principal to demo XRPL borrower wallet; record agent identity and signed task mandate.
2 Request credit	Agent requests 2,000 units of demo RLUSD-equivalent, 90 days, 5.5% APR for working capital.
3 Attach Gita evidence	Gita returns verified issuer identity, maturity/repayment history, coverage and known exposure; not a credit approval.
4 Underwrite independently	Lender A evaluates existing policy and issues a 2,000 loan offer against its facility; Lender B declines by its own policy.
5 Compare offers	Nick converts lender offer and principal mandate to a canonical schema, finds overlap, returns a permitted envelope.
6 Authorize exact action	Shellfish checks identities, signatures, exact fields, nonce, freshness, available facility headroom and a required signing route.
7 Execute / validate	Backend constructs LoanSet; Loan Broker and borrower sign according to supported XRPL flow; adapter submits and waits for validation.
8 Reconcile / learn	Update ledger-linked loan state and facility reservations; a subsequent LoanPay changes outstanding exposure and adds performance evidence.

3.2 Diagram — responsibility and control flow

FROM	HANDOFF	TO
Principal + agent	Signed borrowing mandate + credit intent	Gita evidence / request orchestrator
Gita	Versioned evidence, provenance, known obligations	Lender A / Lender B underwriting adapters
Each lender	Approve/decline; signed lender-specific offer	Nick normalized policy matching
Nick	Bounded compatible offer + source hashes	Shellfish authorization kernel

FROM	HANDOFF	TO
Shellfish	ALLOW/DENY, reservation, canonical transaction hash	Required borrower/broker controlled signer
Controlled signer	Authorized signed LoanSet or denial	XRPL lending adapter
XRPL validated state	Loan/repayment tx state and result	Exposure registry + Gita + receipts

Boundary note: Lender-owned risk decisions happen before Shellfish. A controlled signer enforces participation for the enrolled demo wallet; Shellfish cannot prevent unrelated holders from transacting elsewhere.

3.3 Failure paths

SCENARIO	RESPONSE	CLIENT EXPERIENCE
Lender rejects credit	NO_OFFER / UNDERWRITING_DECLINED	Show lender-specific rejection; do not overwrite other lender decisions
No compatible terms	DENY_NO_OVERLAP or DENY_RATE / TERM	Show exact failed constraint and no signable transaction
Agent exceeds mandate	DENY_AMOUNT / DENY_PURPOSE	No reservation or signature
Facility headroom exhausted	DENY_CAPACITY	Return approved maximum or retry after re-evaluation
Expired credential / nonce reused	DENY_EVIDENCE_STALE / DENY_REPLAY	Require refreshed evidence or new authorized attempt
XRPL not validated / failed	SUBMITTED_PENDING → FAILED_RECONCILE	Do not mark loan active; reconcile reservation and show reason

4 Economic and trust model

4.1 Parties, trust and liability

PARTY	CONTROLS	ASSUMED ECONOMIC / LEGAL ROLE
Borrower principal	Borrowing mandate; business authority	Owes repayment subject to executed legal agreements; agent is not standalone borrower
Lender / Loan Broker	Underwriting, approved loan terms, required loan signature	Originates/manages lending and assumes contract-defined credit exposure
Vault depositors / broker	Pooled funding, optional first-loss cover	Losses allocated under the XRPL vault/broker configuration; not automatically borne solely by the lender
Gita / Shellfish service	Evidence transport, policy decisions, receipts	Does not guarantee repayment or backstop default in the MVP
XRPL network	Validates ledger transactions	No opinion on borrower solvency or off-chain legal enforceability

No protocol-level score converts into purchasing power. Facility capacity is an underwriter-issued limit scoped to a lender, asset, borrower, validity period and policy version. The model must not add unrelated lenders' facilities and describe the sum as a universally available credit line.

4.2 Three amounts that must never be conflated

VALUE	MEANING	ILLUSTRATIVE FIXTURE
Known borrower exposure	Observed obligations from participating data sources	3,000 RLUSD; partial coverage only
Lender A approved facility	Lender A's own credit limit based on its policy	5,000 RLUSD, committed 1,000, headroom 4,000
Agent mandate ceiling	Maximum loan the principal authorized its agent to request	2,500 RLUSD, 90 days, ≤600 bps

4.3 Unit economics and trust assumptions

- The system is a B2B infrastructure hypothesis: lenders/loan platforms may pay for policy adapters, auditability and agent signing controls. No pricing, conversion or willingness-to-pay has been validated.
- Creditworthiness requires attributable evidence and actual repayment outcomes. Synthetic repayment records demonstrate API shape, not predictive underwriting quality.
- Lenders retain source acceptance lists and reserve the right to reject evidence or require their own KYC/KYB.
- Borrower legal obligation, collections and suitability are off-chain institutional responsibilities, not guaranteed by LoanSet alone.
- Cross-lender double borrowing cannot be prevented outside participating registries; market-wide debt completeness must never be claimed.

4.4 Primary product success criteria

Demonstrate that a new lender policy adapter can evaluate the same evidence and participate in a controlled borrowing flow without rewriting the core Shellfish logic. Measure integration time, authorization denial accuracy and correct validated reconciliation—not hypothetical increases in TVL or reduced credit losses.

5 Detailed functional requirements

ID	PRIORITY	CAPABILITY	ACCEPTANCE INTENT
FR-01	P0	Identity / principal	Link human/business principal, borrower XRPL account, agent ID and mandate; reject missing mapping.
FR-02	P0	Signed mandate	Verify signer, scope, max amount/APR, asset, purpose, allowed lenders, term, expiry, policy version and nonce.
FR-03	P0	Evidence passport	Return versioned, attributable repayment and exposure data plus freshness and coverage labels; optional tier is never binding.
FR-04	P0	Lender policy evaluation	Each lender independently evaluates evidence and issues an approved offer or precise decline.

ID	PRIORITY	CAPABILITY	ACCEPTANCE INTENT
FR-05	P0	Offer normalization	Normalize exact principal, asset, pricing and term units; reject unsupported or lossy translations.
FR-06	P0	Deterministic match	Intersect eligible lender offer with borrower mandate; no LLM final permissions.
FR-07	P0	Authorization kernel	Reject unauthorized agent, stale evidence, wrong asset, changed transaction, excessive amount/cost, unknown lender or replay.
FR-08	P0	Facility reservation	Atomically reserve lender A headroom with optimistic locking/serializable DB transaction and idempotency.
FR-09	P0	Controlled signing	Only release a required signature for the exact canonical bytes/hash approved by Shellfish.
FR-10	P0	XRPL execution	Submit a compatible native LoanSet; store actual validated result and Loan ID; never infer success from submission.
FR-11	P0	Receipts	Record separate decision, signing, submission, validation and reconciliation transitions with source hashes.
FR-12	P0	Denial demo	Show rate/amount/expiry/replay failures without a signature or XRPL loan.
FR-13	P1	Repayment	Observe or originate a valid LoanPay and update known debt and performance history.
FR-14	P1	Concurrency safety	Prove at most one competing reservation succeeds when facility headroom is insufficient.
FR-15	P1	Human readability	UI displays who approved, what was permitted, what executed, and any missing data.
FR-16	P2	Institution adapter SDK	Add real underwriting APIs and trust policy modules with formal SLAs, credential verification and data governance.

5.1 Explicit lender-policy and compatibility semantics

Lenders first evaluate borrower eligibility using their own evidence rules. Only a lender-approved offer enters Nick's compatibility stage. Match exact APR or feasible APR interval against the borrower's maximum total cost; match feasible loan-size range, maturity/payment schedule, asset issuer/currency, lender allowlist, principal identity and purpose. Do not use "min(lender APR max, borrower APR max)" to imply a lender will accept a lower-than-minimum rate. Fees and all-in borrower cost must be accounted for separately.

5.2 Design constraints for AI involvement

- An LLM may explain, extract or propose. Canonical policy schemas are independently validated by deterministic code.
- The final permitted envelope is derived only from lender-signed/approved inputs, borrower-signed mandate, and sufficiently fresh evidence.
- The agent never has unrestricted access to borrower/broker master seeds in the demo architecture.
- Agent transcripts and suggestions are not themselves credit authorizations or ledger execution proofs.

6 Canonical data model & sample payloads

All examples below are illustrative, synthetic and subject to network-specific SDK validation. Money is represented as decimal strings with explicit asset issuer/scale; percentages use basis points in application policy objects. XRPL transaction fields may use different units.

6.1 Evidence passport (Gita)

```
{
  "subject_id": "business:harbor-supply-demo", "issuer_id": "demo-attestor",
  "issued_at": "2026-10-24T18:00:00Z", "expires_at": "2026-10-25T18:00:00Z",
  "evidence_version": "ev-12", "signature": "sig-demo-attestor",
  "history": {"on_time": 8, "late": 1, "default": 0},
  "known_exposure": {"asset": "RLUSD", "amount": "3000.00",
    "coverage": "participating_sources_only", "as_of": "2026-10-24T18:00:00Z"},
  "optional_tier": "A_DEMO", "evidence_hash": "sha256:..."
}
```

6.2 Principal-issued borrowing mandate

```
{
  "mandate_id": "m-001", "principal_id": "business:harbor-supply-demo",
  "agent_id": "agent:procurement-01", "xrpl_borrower": "rDEMO...",
  "asset": {"currency":"RLUSD", "issuer":"rISSUER..."},
  "max_principal": "2500.00", "max_apr_bps": 600,
  "max_total_fees": "30.00", "max_term_days": 90,
  "allowed_purposes": ["working_capital"], "allowed_lenders": ["lender-A"],
  "expires_at": "2026-10-24T23:00:00Z", "nonce": "unique-mandate-01",
  "policy_version": "borrower-v1", "principal_signature": "sig-demo-principal"
}
```

6.3 Lender-issued facility and firm offer

```
{
  "lender_id": "lender-A", "borrower_id": "business:harbor-supply-demo",
  "facility_id": "facility-A-001", "facility_limit": "5000.00",
  "active_commitments": "1000.00", "reserved": "0.00",
  "offer_id": "offer-A-777", "asset": "RLUSD", "principal": "2000.00",
  "apr_bps": 550, "term_days": 90, "fees": "0.00",
  "purpose": "working_capital", "expires_at": "2026-10-24T22:00:00Z",
  "underwriting_policy": "lenderA-v2", "approval_ref": "uw-A-349",
  "lender_signature": "sig-demo-lender"
}
```

6.4 Compatibility + authorization receipt

```
{
  "request_id": "req-003", "compatible": true,
  "offer_id": "offer-A-777", "mandate_id": "m-001",
  "proposal_hash": "sha256:canonical-proposal", "envelope_hash": "sha256:...",
  "decision": "ALLOW", "reason_codes": [],
  "checks": {"identity":"PASS", "lender_offer":"PASS",
    "policy":"PASS", "exposure":"PASS", "nonce":"PASS"},
  "reservation_id": "res-A-999", "signature_state": "NOT_YET_SIGNED",
  "xrpl_state": "NOT_SUBMITTED", "tx_hash": null,
  "decision_time": "2026-10-24T18:11:00Z"
}
```

A receipt's decision and transaction fields must advance as independent events; never fill tx_hash with a fabricated value. Hashes use canonical serialization with versioned schemas. A tier label does not change facility headroom, and a repayment event does not automatically expand lender-approved limits.

7 API contracts & backend service design

7.1 P0 API endpoints

ENDPOINT	CODE OWNER	MUST RETURN / SIDE EFFECT
POST /v1/mandates	You — Shellfish module	Store signed mandate and immutable policy version; identity-binding result
GET /v1/evidence/{id}	Gita — evidence module	Issuer-signed evidence passport with coverage, freshness and history
POST /v1/lenders/{id}/evaluate	Gita — Nick spec reviewed	APPROVE or DECLINE; facility/offer refs; policy version
POST /v1/offers/match	Gita — Nick spec reviewed	Matched offer(s), permitted terms or DENY_* reason
POST /v1/authorizations	You — Shellfish module	ALLOW/DENY, reason codes, proposal/envelope hashes
POST /v1/loans/originate	You — XRPL/orchestrator	Idempotent reservation → controlled sign → submit → reconcile
GET /v1/loans/{id}	You — XRPL/orchestrator	Local workflow state + verified on-chain Loan ID if available
GET /v1/receipts/{id}	You — receipt service	Decision, signatures, tx status, timestamps, evidence refs
POST /v1/demo/repay	You (XRPL) + Gita (evidence)	Submit actual LoanPay or synthetic event explicitly flagged

7.2 Suggested storage schema

TABLE	KEY FIELDS / INTEGRITY CONSTRAINT
principals	principal_id, verified_by, legal_name_stub, borrower_xrpl_account, status
agents	agent_id, principal_id, key_id, delegated_scope, disabled_at

TABLE	KEY FIELDS / INTEGRITY CONSTRAINT
mandates	mandate_id PK, principal_id, version, signature, expires_at, immutable_hash
evidence_passports	evidence_id PK, subject_id, issuer_id, version, as_of, expiry, hash, coverage
facilities	facility_id PK, lender_id, borrower_id, approved_limit, committed, reserved, asset, version
offers	offer_id PK, facility_id, apr_bps, fees, principal, term, approved_ref, expiry, signed_hash
reservations	reservation_id PK, facility_id, proposal_hash UNIQUE, amount, state, expires_at
authorizations	decision_id PK, nonce UNIQUE (scoped), proposal_hash, envelope_hash, status, reason
loan_executions	execution_id, decision_id, xrpl_tx_hash UNIQUE NULLABLE, LoanID NULLABLE, validated_status
loan_events	loan_id, event_id UNIQUE, type, amount, ledger_index, evidence_eligible, event_time

7.3 State machine and failure recovery

```
REQUESTED -> UNDERWRITING -> MATCHED -> AUTHORIZED -> RESERVED
  -> SIGNED -> SUBMITTED -> VALIDATED_ACTIVE -> PARTIAL/REPAID
                                             -> DELINQUENT/DEFAULTED
  -> DENIED (from validation stages)
  -> SUBMISSION_FAILED/EXPIRED -> RECONCILING -> RELEASED or ACTIVE
```

A successful DB reservation is not a successful loan. State changes must be idempotent on request_id + facility_id + proposal_hash. On submission timeout, query XRPL transaction status before releasing headroom; do not blindly retry with a new nonce. The reservation is consumed into committed exposure only on validated success. Persist failed/reconciled receipts with the original denial or ledger codes.

7.4 Service boundaries and build stack (proposal)

SERVICE	STACK	SCOPE
API / orchestrator	FastAPI or TypeScript/Node; one shared REST backend	Auth, sequencing, persistence, response schemas
Policy layer	Pure TypeScript or Python package	Schema validation and deterministic checks with unit tests
Registry	SQLite (demo) or PostgreSQL	Transactional facility headroom reservation, idempotency
XRPL adapter	xrpl.js + Devnet WebSocket	Create loan, sign in required order, submitAndWait, inspect metadata
Frontend	React + simple component system	One timeline and approvals/denials; link real transactions

8 XRPL integration specification

8.1 Native primitives and what we do not rebuild

PRIMITIVE	ROLE	INTEGRATION NOTE
XLS-65 / Vault	Single-asset pooled liquidity	Vault setup prerequisite, not a Shellfish-built liquidity pool
LoanBroker	Manages native loan origination and cover configuration	Existing broker config or tutorial setup, with verified ID
LoanSet	Originate credit with both broker and borrower consent	Two required signatures; exact terms from authorization
LoanPay	Repayment from borrower	Only record real repayment after validated successful transaction
LoanManage	Impair or default a native loan	Not a routine MVP action; simulate servicing cases where needed

8.2 First three-hour network spike

1. Call XRPL public feature API on the chosen endpoint; check SingleAssetVault, LendingProtocol and, where required by the SDK/tutorial, LendingProtocolV1_1. Never infer availability from a documentation snippet alone. [R4]
2. Run the official lending tutorial setup on an applicable Devnet endpoint with a vault, loan broker, prefunded borrower and required asset. [R3]
3. Perform the smallest manual two-signature LoanSet outside Shellfish first; require a validated tesSUCCESS and inspect the created Loan object.
4. Record endpoint, observed feature status, network, asset representation, transaction hash and whether LoanPay can execute. Freeze go/no-go by hour 3.

8.3 Signing boundary: exact path

LoanSet uses borrower and broker signatures, including a counterparty signature in the supported transaction structure. The demo must place at least one necessary signer behind Shellfish (preferably the borrower's delegated signing service; optionally the participating broker signer as well). The signed bytes must match the canonical proposal hash and approved offer reference. If the agent independently holds unrestricted borrower keys, the security claim fails. [R3, R5]

Application APR uses basis points (550 = 5.5%). The current XRPL tutorial encodes LoanSet InterestRate in tenths of a basis point (5500 = 5.5%); adapter conversion must be explicit and unit-tested. PaymentInterval uses seconds; example 30-day interval is 2,592,000 seconds. [R3]

8.4 Environment-dependent fallback

HONEST DEMO-MODE CONTRACT

LIVE_DEVNET: Show actual LoanSet, validation and Loan ID. SIMULATED_LOAN: Show signed request + authorization and deterministic simulated loan state, alongside any real XRPL payment only if clearly labeled as distinct. Never label a simulated loan "on-chain lending" or imply Mainnet readiness.

8.5 Telemetry and verified outcomes

- Observe result meta.TransactionResult, validated status, ledger index and created Loan object ID; save tx hash with network ID.
- Pin transaction request to a reasonable LastLedgerSequence; on uncertain submission, query chain before retry.
- Distinguish SIGNATURE_ISSUED, SUBMITTED, VALIDATED_ACTIVE, FAILED and RECONCILING; no implicit promotion to success.
- Explicitly report service availability, network feature flags and simulation mode in UI.

9 Security, abuse & privacy requirements

THREAT	REQUIRED MVP CONTROL	KNOWN LIMITATION
Agent mandate inflation	Principal signature, immutable mandate hash, strict bounds	Cannot constrain agents operating outside enrolled signer
Replay / duplicate issuance	Per-principal nonce, idempotency keys, transaction hash binding	Distributed deployments need consistent shared DB
Facility race condition	Atomic DB reserve with version/CAS + unique reservation	No global guarantee across unaffiliated lenders
Stale evidence / rescinded offer	Issuer trust allowlist, expiry, approval reference/version	Simulated issuers do not validate real identity
Score washing / fabricated history	Signed provenance + raw outcome refs; no binding score	Fraud detection needs trusted external sources
Key leakage	Secrets out of agent prompt, controlled signer, no log secrets	Demo signer is not production HSM
Policy drift / unit mismatch	Canonical schema and property-based tests for rates, asset IDs	Adapters still need audits for live integrations
Privacy / data leakage	Off-chain records; minimal hashed refs in receipts	Hashing alone is not anonymity
Approval bypass	Required signer checks canonical exact tx on every attempt	Cannot prevent unrelated wallet contracts or brokers
Disputed debt / legal rights	Only fictional demo principal; contractual assumptions explicit	Real-world enforceability/compliance out of scope

9.1 Canonical denial codes

DENY_IDENTITY, DENY_SIGNATURE, DENY_SCOPE, DENY_ASSET, DENY_AMOUNT, DENY_RATE, DENY_FEE, DENY_TERM, DENY_PURPOSE, DENY_COUNTERPARTY, DENY_OFFER_EXPIRED, DENY_EVIDENCE_STALE, DENY_CAPACITY, DENY_REPLAY, DENY_NO_OVERLAP, DENY_TRANSACTION_MISMATCH, and DENY_UNSUPPORTED_NETWORK. Return stable machine-readable codes plus one human explanation; never expose another participant's private reservation thresholds.

9.2 Security acceptance tests

CASE	EXPECTED RESULT
Valid approved 2,000; 5.5% APR; 90 days	ALLOW; controlled signatures; submit/validated or labeled simulation
Borrow 3,000 against agent max 2,500	DENY_AMOUNT; no signature, no reservation
Offer 9% against max 6% APR	DENY_RATE; no signing
Replay same nonce and proposal	DENY_REPLAY; never second loan
Untrusted evidence issuer or expired bundle	DENY_EVIDENCE_STALE / DENY_IDENTITY
Swap XRPL asset issuer after approval	DENY_TRANSACTION_MISMATCH / DENY_ASSET
Race: two 2,500 requests against 4,000 headroom	At most one reservation wins; other DENY_CAPACITY
Execution timeout after signature	RECONCILING; do not prematurely release or mark active
Full valid repayment	Update debt outstanding and history; not lender facility ceiling

10 Frontend and judge-facing demo

10.1 Three-screen experience

SCREEN	INFORMATION / INTERACTION
A — Borrower workspace	Business principal, delegated agent, signed max 2,500, purpose, term, cost cap, requested 2,000
B — Credit & lender decisions	Gita source/coverage, lender A approve at 5.5% / lender B decline with distinct policy rules, Nick compatible envelope
C — Authorization & XRPL receipt	Shellfish check matrix, reservation, signer status, validated hash, Loan ID, repayment event; one DENY attempt

10.2 Example seeded fixture

ENTITY	ILLUSTRATIVE RULE
Borrower: HarborSupply (fictional)	Demo account with 6 months evidence, 3,000 known network exposure
Lender A	Approves 2,000 RLUSD at 550 bps / 90 days; facility 5,000, 1,000 committed
Lender B	Requires ≥12 months accepted history; DECLINE despite the same evidence
Agent mandate	Max 2,500 principal, max 600 bps, 90 days, working-capital purpose, Lender A only
Expected success	2,000 at 550 bps: COMPATIBLE → ALLOW → signing → validated loan
Expected failure	3,000: DENY_AMOUNT; 9%: DENY_RATE; replay: DENY_REPLAY

10.3 90-second demo script

5. “XRPL gives institutions loan settlement but intentionally leaves underwriting and delegated-agent controls off-chain.”
6. Open borrower workspace; show the signed company mandate. “The AI may search, but cannot grant itself credit authority.”
7. Open Gita passport; explain issuer provenance and “known exposure,” not a universal score.
8. Run two lender adapters; show one approves and one declines using existing risk policies.
9. Nick chooses a compatible offer; Shellfish authorizes exact terms, facility reserve and signer.
10. Show Devnet LoanSet validation and a receipt; submit unauthorized change and display deterministic denial.
11. Finish with repayment evidence (actual if available; otherwise labeled simulation). “The lender decides. Shellfish enforces. XRPL settles.”

10.4 What a judge should remember

JUDGE-READY ONE-LINER

Shellfish lets lenders keep their underwriting and lets AI agents access XRPL credit only within verified, lender-approved and owner-delegated limits.

11 36-hour delivery plan & team interfaces

TIME	YOU — CODE (XRPL + SHELLFISH + UI)	GITA — CODE (EVIDENCE + POLICIES)	NICK + SHELL — NON-CODING
0–3h	Prove native Devnet loan path; establish signing boundary, framework and interfaces.	Freeze evidence, lender-offer and compatibility schemas; bootstrap fixtures and shared DB.	Nick: approve lender-policy rules and sample cases. Shell: define required authorization and abuse checks.
3–10h	Build mandate verification, Shellfish decision/receipt API and signer integration skeleton.	Build Gita passport API, 2 deterministic lender adapters and Nick compatibility logic.	Nick: review match semantics and examples. Shell: review signing assumptions and edge-case matrix.
10–18h	Implement XRPL loan adapter, controlled signer, authorize-to-submit orchestration.	Add transactional lender-specific facility reservation, evidence and policy tests; expose stable endpoints.	Nick: check pricing, term and decline explanations. Shell: challenge replay/expiry/bypass behavior.

TIME	YOU — CODE (XRPL + SHELLFISH + UI)	GITA — CODE (EVIDENCE + POLICIES)	NICK + SHELL — NON-CODING
18–26h	Build three-screen demo UI and integrate happy path + one denial; validate real or simulated status.	Integrate repay/closure evidence, reservation reconciliation and API regression suite.	Nick: draft pitch/demo story. Shell: run manual abuse tests; log issues for developers.
26–31h	Connect final validated hash or prominently labeled loan simulation; harden UI and demo command.	Fix integration bugs, verify evidence update and denial reasons; support demo playback.	Nick: finalize written submission and demo narration. Shell: final security and claims review.
31–36h	Code freeze except critical defects; run full demo, record video and submit repository.	Run clean-slate smoke tests and fixture replay; assist last-minute P0 defect fixes.	Nick: lead pitch rehearsal + judging answers. Shell: verify controls, disclaimers and Q&A.

11.1 Milestone gates

- H03: native lending go/no-go chosen; schema interfaces frozen.
- H10: Gita, lender policy and Shellfish APIs independently return fixture-correct payloads.
- H18: full authorized flow works locally, using actual signing integration or clearly controlled stub.
- H26: end-to-end demo works with one allow and three denials, with evidence and receipt.
- H31: live XRPL hash or simulation badge shown correctly; no scope expansions.
- H36: README, repo, video/presentation and deterministic fixture script ready.

11.2 Proposed repo layout

```
shellfish/
  apps/web/                 # Borrower + lender + receipt UI
  services/api/            # HTTP orchestration + storage
  packages/schemas/        # Mandate, evidence, facility, offer, receipt
  packages/evidence/       # Gita issuer + known-exposure adapter
  packages/compatibility/  # Nick deterministic normalizer/matcher
  packages/authorization/  # Shellfish mandate verify + policy gate
  packages/xrpl/           # Vault/broker setup + LoanSet/LoanPay
  fixtures/                # Fictional lenders/borrowers/test states
  tests/                   # Core invariants + replay/race tests
  README.md                # Setup, network mode, assumptions, limitations
```

11.3 Definition of done

- A fresh teammate can run the seeded happy path using one command and read real vs simulated modes clearly.
- At least two lender policies evaluate identical evidence to different outcomes.
- A valid request produces an ALLOW receipt and validated loan (when target features support it); failures show no unauthorized signature.
- At least six deterministic negative tests and one idempotency/replay test pass; race test prioritized.

- Every displayed tx hash can be queried against the specified network; no synthetic hash presented as real.
- The demo includes explicit financial, compliance, signer-enforcement and credit-data coverage limits.

11.4 Four-person team ownership — two engineers

Binding staffing assumption: Four teammates participate, but only you and Gita write, review and merge implementation code. Nick and Shell own product/policy and safety validation respectively; they are NOT assigned executable implementation tasks. “Nick” and “Shellfish” still name system modules, but module names do not imply who codes them. This allocation supersedes any earlier three-developer plans.

TEAM MEMBER	CODES ?	RESPONSIBILITY	DELIVERABLE
You (engineer)	YES	Shellfish authorization, signing, XRPL adapter, backend orchestration and demo UI.	Validated Devnet loan or labeled fallback, receipt and denied over-limit request.
Gita (engineer)	YES	Evidence passport, lender policies, Nick matching, atomic facility registry and tests.	Stable APIs, distinct lender decisions and matching/reservation tests.
Nick (product)	NO	Lender policy semantics, offer rules, pricing examples, demo positioning and pitch.	Lender policy matrix, acceptance examples, script and judge Q&A.
Shell (security)	NO	Authorization invariants, signer threat model, security review and manual adversarial QA.	Security checklist, denial scenarios, tested limitations and judge Q&A.

11.4.1 Day-of coordination and interface contract

Hour 0–3 — You verify Devnet lending, key custody and a minimal two-party loan; Gita freezes the request/evidence/offer/compatibility response schemas. Nick provides two lender rules with expected approval and rejection outcomes. Shell provides signing constraints and six negative tests. Agree on API shapes before parallel work.

Hour 3–10 — Gita publishes fixture-backed GET /v1/evidence/{id}, POST /v1/lenders/{id}/evaluate, and POST /v1/offers/match. You publish POST /v1/mandates, POST /v1/authorizations, and the XRPL adapter interface. Each engineer may use local fixtures until the other API is stable; integration consumes the canonical schema, not ad hoc field names.

Hour 10–26 — You integrate Shellfish, required signing and loan execution into one borrower UI. Gita owns transactional facility reserve/release and evidence feedback, so the orchestration must invoke her reservation service before signing. Nick manually reviews lender-policy semantics; Shell manually tests mandate escalation, replay, stale evidence and controlled-signer bypass assumptions. Neither reviewer blocks core engineering work with open-ended scope additions.

Hour 26–36 — Freeze features; two developers run the end-to-end smoke test and fix only P0 failures. Nick owns submission text and live narrative. Shell owns claim/security review and judge-question preparation. All four rehearse the 90-second demo; you are the integration owner and decide which fallback mode is shown.

11.4.2 Two-developer scope protections

- P0: one accountable borrower and signed mandate; two simulated lender policies; Gita passport; one compatible offer; one Shellfish ALLOW and at least three DENY cases; one actual Devnet origination if feasible, otherwise plainly labeled loan simulation with real XRPL receipt only for actual on-chain transactions.
- P0: lender-specific capacity reserve must be atomic inside the demo registry; reserve failures, submission failures and replay must not create duplicate available credit or falsely claim settlement.
- P1 only after H26: full repayment UX, advanced charts, third lender, agent-generated natural-language policy parser, detailed dashboards and additional failure animations.
- Never attempt during the hackathon: real KYC, bank API integration, universal score training, legal lending contracts, cross-lender total debt guarantees, micro-bond secondary trading or production custody.
- No requirement may assume Nick or Shell contribute code. If a code task is unowned, defer it, shrink it to a fixture, or move it explicitly to you or Gita at the next checkpoint.

12 Risks, assumptions & open decisions

ID	ISSUE	DECISION / MITIGATION
R-01	XRPL lending unavailable on selected server	Check feature flags and supported Devnet first; fail over to labeled simulation
R-02	Borrower/broker signing bypass	Make required signer conditional on policy approval; keep keys out of agent process
R-03	Lender approval not trustworthy	Demo adapter only; production lender must sign / authenticate offer
R-04	Exposure registry incomplete	Show “participating sources only”; never advertise total market debt
R-05	Credit history is synthetic	Label demo-only; no claims of predictive accuracy
R-06	Facility headroom not atomic	One central transactional reservation service; test concurrent requests
R-07	Asset availability or token setup	Use supported test token; avoid falsely labeling a token RLUSD
R-08	Legal borrower / responsibility ambiguity	Use verified legal principal assumption; real lending contracts outside MVP
R-09	Scope inflation to bonds / secondary trading	Explicitly excluded; no NFTs, obligations marketplace or custom loan primitive
R-10	No commercial buyer validation	Present B2B integration hypothesis, not proven revenue or TVL impact

12.1 Decisions to confirm by hour 3

12. Which Devnet endpoint and which exact amendments are enabled?
13. Which Loan Broker and borrower accounts are used, and who owns each required signing key?
14. Which demo asset/issuer is configured? Is it genuine RLUSD or only a RLUSD-like test asset?
15. Confirm engineering split: You own XRPL adapter, Shellfish signing/authorization, frontend integration and recording; Gita owns evidence, lender adapters, compatibility, facility registry and test fixtures.
16. Do we have a valid lender-signed approval fixture, or is the lender adapter openly simulated?

12.2 Post-hackathon roadmap

- V1 — Production-signing design: HSM/MPC, revocation, strong KYC/KYB, enterprise audit, security review.
- V2 — Lender integrations: standard interface to existing loan origination/risk engines, consent-driven evidence providers, digital attestations.
- V3 — Verified portable histories: selective disclosure, provenance standards, institution trust lists, multi-lender exposure reconciliation.
- V4 — Optional agent quote negotiation and additional machine-payment ingress adapters; maintain deterministic permission gates.

13 External references & source provenance

The product architecture follows the team's 24 October "Portable Credit Capacity & Bounded Agent Lending on XRPL" specification, corrected by the subsequent decision to preserve lender-owned underwriting and abandon micro-bond secondary trading. The micro-bond brief is used as a boundary document only; its transfer/negotiation marketplace is explicitly out of scope.

REF	SOURCE / USE
[I1]	Team concept: "XRPL Portable Credit Capacity / Agent Lending Architecture," 24 Oct 2026. Evidence, lender policy, compatibility, authorization, exposure and receipts.
[I2]	"Shellfish Micro-Bond Protocol Concept Brief," 24 Oct 2026. Rejected as the chosen 36-hour product; retained as possible later separate extension.
[R1]	XRPL docs — Lending Protocol. https://xrpl.org/docs/concepts/tokens/lending-protocol
[R2]	XRPL docs — Single Asset Vaults. https://xrpl.org/docs/concepts/tokens/single-asset-vaults
[R3]	XRPL docs — Create a Loan tutorial. https://xrpl.org/docs/tutorials/defi/lending/use-the-lending-protocol/create-a-loan
[R4]	XRPL docs — feature API & Known Amendments. https://xrpl.org/docs/references/http-websocket-apis/public-api-methods/server-info-methods/feature ; https://xrpl.org/resources/known-amendments
[R5]	XRPL standard — XLS-0066 Lending Protocol (draft). https://xls.xrpl.org/xls/XLS-0066-lending-protocol.html
[R6]	XRPL docs — LoanPay. https://xrpl.org/docs/references/protocol/transactions/types/loanpay

END OF PRD • INTERNAL HACKATHON BUILD CONTRACT • ALL EXAMPLE IDENTITIES AND CREDIT DATA ARE FICTIONAL